AI-Powered Intelligent Query Resolution System

1. Introduction

Organizations hold a huge amount of knowledge in handbooks, HR policies, manuals, FAQs and process guides. The problem is not missing knowledge but finding it quickly. People end up keyword-searching, scrolling through PDFs, or waiting for a human to reply, which causes delays and inconsistent answers.

This project builds a **domain-agnostic, voice- and text-enabled conversational platform**. A user uploads any knowledge base (PDF, DOCX, TXT or FAQ CSV) and can query it in natural language straight away. The system automatically builds a **Retrieval-Augmented Generation (RAG)** pipeline and runs a **multi-agent resolution layer** on top of it.

RAG: A plain LLM can hallucinate. RAG makes it first retrieve relevant chunks from the uploaded documents and then generate the answer only from those chunks, so every answer is grounded and traceable.

Why multi-agent? One monolithic prompt handles real queries badly. Splitting the work into specialised agents (understand, retrieve, answer, clarify, remember) makes each step easier to build, test and improve.

Key differentiators

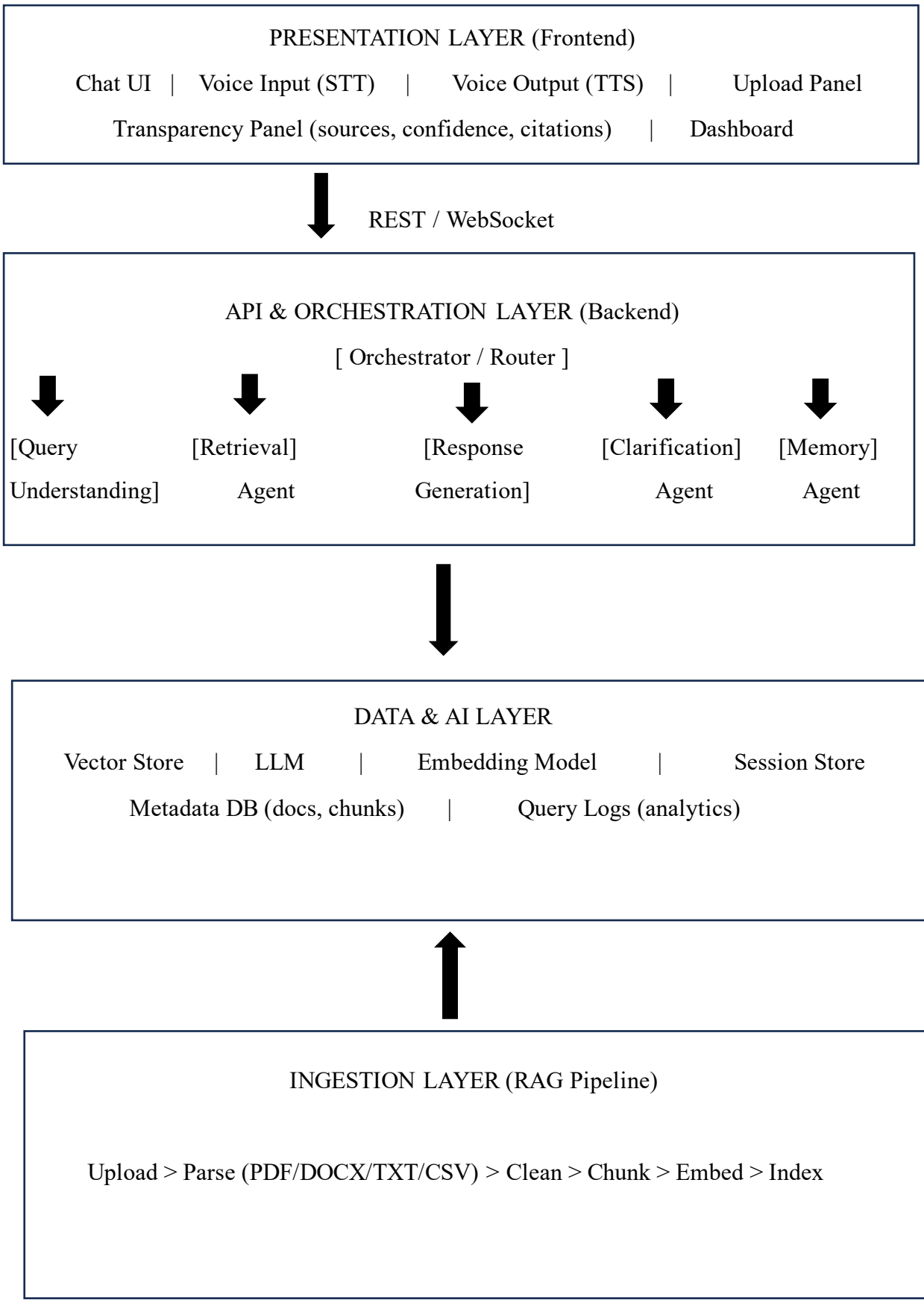
- Works for any domain (college handbook, company HR policy, technical docs) with no code changes
- Voice in and voice out through the browser-native Web Speech API
- Full transparency: source chunks, confidence score and citations on every answer
- Multi-turn memory for follow-up questions
- Analytics that reveal knowledge gaps, meaning questions the knowledge base could not answer

Objectives

1. Develop a domain-agnostic RAG pipeline that ingests any knowledge base.
2. Build a multi-agent layer: Query Understanding, Retrieval, Response Generation, Clarification and Conversation Memory.
3. Enable voice input and text-to-speech output via the Web Speech API.
4. Provide response transparency: source chunks, confidence scoring and citations.
5. Support multi-turn conversations with contextual memory.

2. System Architecture

2.1 Layered view



2.2 A. Presentation layer

- The **chat interface** shows the multi-turn conversation.
- The **voice module** uses the Web Speech API. SpeechRecognition converts speech to text and SpeechSynthesis reads answers aloud. Both run in the browser, so no external speech service is needed.
- The **transparency panel** shows, for every answer, the retrieved source chunks, a confidence indicator and citations (document name, page or section).
- The **analytics dashboard** shows query trends and knowledge gaps.

B. Ingestion layer (Module 1)

1. **Upload and parse:** extract text from PDF, DOCX, TXT and CSV. For CSV FAQs, each Q&A row becomes one unit.
2. **Clean:** strip headers, footers and noise.
3. **Chunk:** split into overlapping segments (for example 500 to 800 tokens with 10 to 15 percent overlap) so context is not lost at boundaries. Metadata such as source file, page and section is kept.
4. **Embed:** convert each chunk into a vector with an embedding model.
5. **Index:** store vectors and metadata in a vector database such as FAISS or ChromaDB.

Because this pipeline is generic, the same code handles any domain, which is what makes the system domain-agnostic.

C. The five agents

Agent	Responsibility	Input and output
Query Understanding	Detects intent, classifies the query as factual, procedural, comparative or ambiguous, and decides routing	Raw query plus history, giving intent, type, complexity and route
Retrieval	Semantic search in the vector store, then ranking and filtering by relevance (optionally reranking and query rewriting)	Refined query, giving top-k chunks with similarity scores
Response Generation	Writes a grounded answer using only the retrieved chunks, adds citations and computes a confidence indicator	Query plus chunks, giving answer, citations and confidence
Clarification	Detects vague or multi-part queries and asks targeted follow-up questions	Ambiguous query, giving clarifying question(s)

Conversation Memory	Stores turns, resolves references such as "what about that policy?" and supplies context to the other agents	Session history, giving a contextualised query or summary

D. Orchestrator (Module 2)

The control layer. It receives each query, calls agents in the right order and applies routing rules, for example: ambiguous goes to the Clarification Agent, otherwise Retrieval then Generation. It can be built with LangGraph, LangChain or a custom state machine.

E. Data layer

- **Vector store:** semantic index of chunks
- **Metadata DB:** documents, chunks and sessions
- **Query log store:** every query, confidence score and outcome, which feeds Module 6

F. Analytics and knowledge-gap detection (Module 6)

Queries with low retrieval scores, low confidence or negative feedback are flagged and clustered. This shows administrators which topics are missing from the knowledge base.

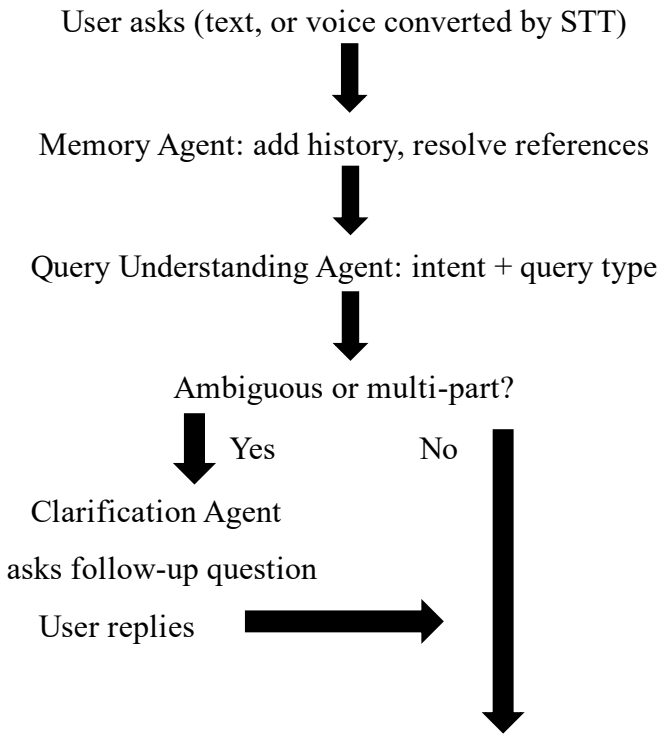
3. Workflow Model

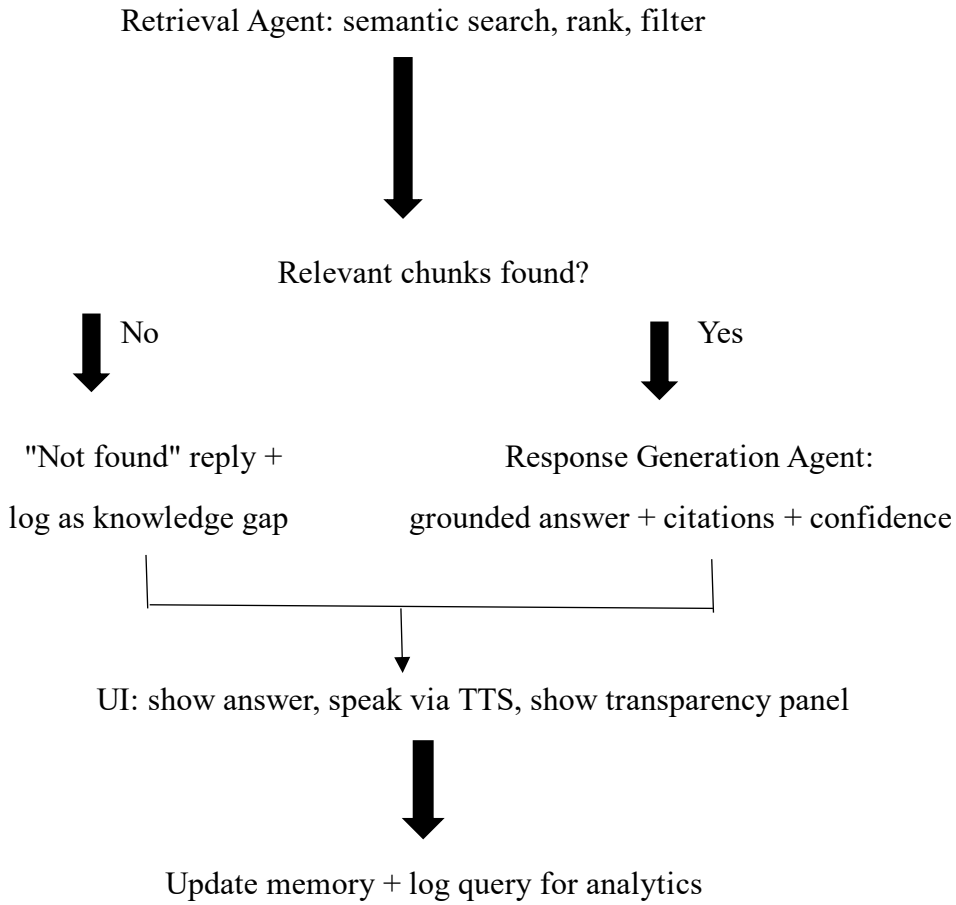
3.1 Phase 1: Knowledge base setup (once per upload)

Upload file > Parse text > Clean > Chunk (overlap + metadata)

> Generate embeddings > Store in vector DB > "Knowledge base ready"

3.2 Phase 2: Query resolution (every user query)





3.3 Step-by-step walkthrough

Example query in an HR-policy knowledge base: "How many leaves can I take?"

1. **Input:** the user speaks and the browser converts it to text.
2. **Memory:** there is no prior context, so the query passes through unchanged.
3. **Understanding:** it is classified as factual, but the leave type is not specified, so it is flagged as ambiguous.
4. **Clarification:** the agent asks, "Do you mean casual, sick or earned leave?"
5. **User replies:** "Sick leave." Memory merges this into "How many sick leaves can an employee take per year?"
6. **Retrieval:** the top 5 chunks are fetched from the leave-policy section, each with a similarity score.
7. **Generation:** the answer states the sick-leave entitlement with a citation such as HR_Policy.pdf, p.14, and the confidence is High.
8. **Output:** the answer is shown and read aloud, and the side panel lists the source chunks.

9. **Follow-up:** the user asks "Can they be carried forward?" Memory resolves "they" to sick leaves and the cycle repeats.
10. **Logging:** all queries and scores are stored, and low-confidence ones surface in the analytics dashboard.

3.4 Confidence scoring (suggested approach)

Combine three signals:

- the average and top similarity scores of the retrieved chunks,
- how many chunks agree with each other,
- whether the LLM's answer is fully supported by the retrieved context.

Map the result to High, Medium or Low. Below a set threshold, the system declines to answer or asks for clarification.

4. Conclusion

The AI-Powered Intelligent Query Resolution System turns static documents into a conversational, trustworthy source of answers. Its RAG pipeline grounds every response in the uploaded knowledge base, which reduces hallucination and keeps answers traceable. Its multi-agent layer (Query Understanding, Retrieval, Response Generation, Clarification and Conversation Memory) splits the work into clear roles, so ambiguous questions are clarified, follow-ups are understood and each step can be tested and improved on its own.

Because ingestion and orchestration are domain-independent, the same platform serves a college handbook, a company HR policy or a technical manual without code changes. Voice input and output through the Web Speech API make it natural to use, while the transparency panel (source chunks, confidence score and citations) builds user trust. The analytics and knowledge-gap module closes the loop by showing administrators what users ask that the knowledge base cannot yet answer, so the content keeps improving.

In summary, the system delivers faster, more consistent and verifiable answers, reduces dependence on manual support, and provides a scalable foundation that can later be extended with multilingual support, user feedback learning, role-based access and integration with enterprise tools such as Slack, Teams or helpdesk systems.